

설비 고장을 예측하는 AI 데이터 분석

결측치 처리

결측치 제거와 대체

02

결측치 제거와 대체

제거로 덜고 대체로 채우는 결측 처리 워크플로우

결측치 제거란 무엇인가

제거는 결측이 있는 행·열을 통째로 지우는 단순하고 정직한 방법

01 · 작동 방식

빈 칸을 가진 줄을 통째로 삭제

**결측 있는 행·열을
통째로 삭제**

판다스 dropna 한 줄로 해결

02 · 장점

추측이 없어 정직함

**없는 값을 지어내지 않는
왜곡 없는 처리**

남은 데이터는 모두 진짜 측정값

03 · 단점

멀쩡한 값도 함께 버려짐

**센서 하나만 비어도
행 전체 손실**

데이터가 충분할 때 쓰는 방법

dropna로 결측 행 삭제

옵션 없는 dropna는 결측이 하나라도 있는 행을 모두 삭제 — 250행 → 76행

CODE

```
df = pd.read_csv("secom_교육샘플.csv")
print(df.shape) # (250, 22)

clean = df.dropna() # 결측 있는 행 삭제
print(clean.shape) # (76, 22)
```

결측이 흩어져 있으면 기본 dropna 한 번에 데이터의 상당 부분이 사라짐
dropna는 원본을 바꾸지 않고 결과를 돌려주므로, **반드시 새 변수에 담아야** 반영됨

axis로 결측 열 삭제

axis=1은 결측 있는 열을 삭제 — 22열 → 10열 (결측 없는 컬럼만 남음)

CODE

```
clean = df.dropna(axis=1) # 결측 있는 열 삭제
print(clean.shape) # (250, 10)

# 특정 열만 콕 집어 빼기
df.drop(columns=["센서19", "센서20"])
```

axis=0은 세로 방향 행, axis=1은 가로 방향 열

실무에서는 통째로 지우는 axis 방식보다 drop으로 센서19·20만 콕 집어 빼는 방식이 더 자주 쓰임

how로 삭제 기준 정하기

같은 dropna여도 how 옵션 하나로 결과가 극과 극 — any 76행 vs all 250행

01 · ANY

어느 하나라도 결측이면

**결측 1개라도 있으면
행 전체 삭제 (기본값)**

SECOM 결과: 174행 삭제 → 76행

02 · ALL

모든 값이 결측일 때만

**전부 비어 있을 때만
행 삭제**

SECOM 결과: 거의 그대로 250행 유지

03 · 활용

실전 사용 패턴

**all로 완전 빈 행만
먼저 정리**

남은 결측은 thresh·대체로 다듬기

thresh로 임계값 삭제

thresh는 살아 있는 값의 **최소 개수**를 정함 — 결측 개수가 아닌 값의 개수 기준

01 · 의미

값이 몇 개 이상이면 살린다 — 방향에 주의

**thresh = 살아 있는 값의
최소 개수 (임계값)**

"결측 N개 이상이면 삭제"가 아니라
"값 N개 이상이면 유지"의 방향

any와 all의 중간, 균형 잡힌 기준선

02 · SECOM 적용

22개 컬럼 중 thresh=20

**값이 20개 이상,
결측 2개 이하인 행만 162행 유지**

기본 dropna(76행)보다 두 배 넘게 살림
너무 부실한 행만 거르는 균형점

df.dropna(thresh=20)

subset으로 특정 컬럼 기준 삭제

중요한 컬럼만 기준으로 삼는 정밀 제거 — **정답 컬럼이 빈 행만** 골라 삭제

01 · 의미

부분집합 기준

**지정한 컬럼만 보고
결측 판단**

다른 컬럼의 결측은 신경 쓰지 않음

02 · 정답 컬럼 보호

대표적 활용 사례

**불량여부가 빈 행만
정확히 삭제**

함부로 채우면 안 되는 정답은 제거가 안전

03 · CODE

여러 컬럼도 지정 가능

```
df.dropna(  
    subset=["불량여부"]  
)  
  
# 여러 컬럼도 가능  
subset=["센서09", "불량여부"]
```

여러 컬럼 지정 시 그중 하나라도 빈 행이 삭제 대상 — 한 잣대가 아닌 컬럼별 정밀 제거

제거의 장단점

SECOM 기본 제거는 **70% 데이터 손실** — 절반 아래로 줄면 멈추고 대체 고려

구분	내용	의미
✅ 장점	간단하고 정직, 추측 왜곡 없음	코드 한 줄로 처리, 남은 데이터는 모두 진짜 측정값
❌ 단점	멀쩡한 값까지 함께 손실	SECOM 250행 → 76행 (70% 손실), 분석 신뢰도·모델 성능 저하
⚠️ 주의	편향 발생 위험	결측이 특정 상황에 몰리면 그 상황 데이터가 통째로 사라짐

제거 후 데이터가 절반 아래로 줄면 멈춰서 다시 생각, 제거 대신 대체를 고려

삭제 전후 shape 검증

확인으로 시작해 확인으로 끝나는 처리의 마무리 — `shape` · `isna` 두 단계

STEP 1

원래 크기 기록

삭제 전 shape로 원래 행·열 개수 확인. 비교의 기준선이 됨.

```
df.shape → (250, 22)
```



STEP 2

줄어든 양 확인

삭제 후 shape와 비교. 의도와 다르게 줄었으면 옵션 점검의 경고등.

```
clean.shape → (76, 22)
```



STEP 3

결측 0 검증

`isna`로 결측을 세어 모두 0이면 처리 완료. 0이 아니면 다시 점검.

```
clean.isna().sum().sum() → 0
```

크기 변화와 결측 잔여, 두 가지를 함께 확인하는 게 처리의 마무리

행 삭제 vs 열 삭제 판단

핵심 판단 기준은 결측이 어디에 몰려 있는가 — 행 집중인가 컬럼 집중인가

결측 분포	판단	이유
일부 행에 집중	행 삭제	측정 실패한 몇몇 행만 빼면 나머지는 거의 살림
특정 센서에 집중	열 삭제	절반 넘게 빈 센서 하나만 빼면 모든 행을 살릴 수 있음
실무 권장 순서	열 → 행	결측 심한 컬럼 먼저 정리 후 부실 행 정리

앞 시간의 결측 비율이 큰 도움 — 센서19·20은 열 삭제 후보, 부실 행은 행 삭제
순서를 바꾸면 손실이 커짐

행 삭제와 열 삭제, 단계적으로

제거는 한 번에 끝내지 않음 — 확실한 것부터 단계적으로 다듬기

STEP 1

부실 컬럼 제거

절반 넘게 빈 부실 컬럼을 먼저 제거. 누가 봐도 살릴 가치 없는 것부터.

센서19·20



STEP 2

부실 행 제거

거의 다 빈 부실 행을 제거. thresh로 임계값을 두면 안전.

dropna(thresh=N)



STEP 3

가벼운 결측은 대체

제거 후 남은 가벼운 결측은 이후 학습의 대체로 채움.

fillna · 평균 · 중앙값

제거와 대체는 경쟁이 아니라 함께 쓰는 동반자 — 너무 적어진다면 미련 없이 대체로 손을 바꿈

실습 1. dropna로 행·열 삭제

결측 있는 행과 열을 삭제하고 크기 변화 확인

목표

결측 있는 행과 열을 삭제하고 크기 변화 확인

단계

- 원본 크기를 shape로 확인
- dropna로 결측 있는 행을 모두 삭제
- 방향을 열로 바꿔 결측 있는 열을 삭제

예상 결과

250×22 → 행삭제 76×22, 열삭제 250×10

실습 2. dropna 옵션 조절

how·thresh·subset로 삭제 기준을 세밀하게 조절

목표

how·thresh·subset로 삭제 기준을 세밀하게 조절

단계

- how로 완전히 빈 행만 삭제하는 기준 적용
- thresh로 값이 일정 개수 이상인 행만 남기기
- subset으로 특정 컬럼이 빈 행만 삭제

예상 결과

완전 결측 행만 삭제는 거의 유지, 임계값 20은 162행

실습 3. 결측 비율 기준 컬럼 제거

결측 비율이 높은 컬럼만 골라 제거

목표

결측 비율이 높은 컬럼만 골라 제거

단계

- 컬럼별 결측 비율을 계산
- 비율이 기준을 넘는 컬럼 이름만 목록으로 뽑기
- 그 컬럼들을 drop으로 제거하고 크기 확인

예상 결과

40% 초과 센서19·20 제거 → 250×20

실습 4. 삭제 손실 비교

삭제 방식별 남는 행 수와 손실률을 표로 비교

목표

삭제 방식별 남는 행 수와 손실률을 표로 비교

단계

- 원본·행삭제·thresh 각 방식의 남는 행 수 구하기
- 방식과 행 수를 하나의 표로 모으기
- 원본 대비 손실률을 백분율로 계산해 나란히 보기

예상 결과

행삭제 손실 약 70%, thresh 손실 약 35%

결측치 대체란 무엇인가

대체는 빈 칸만 콕 집어 채워 데이터를 버리지 않고 **원래 크기 유지**

01 · 작동 방식

제거와 정반대 접근

**빈 칸만 콕 집어
그렇듯한 값으로 채움**

하나도 안 버리고 원래 크기 유지

02 · 쓰는 이유

데이터가 아깝기 때문

**제거하면 70% 손실되는
SECOM에 거의 필수**

빈 값 하나만 채우면 나머지 다 살림

03 · 주의점

공짜는 아님

**채운 값은 추측
너무 많이 채우면 위험**

진짜 데이터 모습에서 멀어질 수 있음

fillna로 상수 채우기

결측이 특정 의미를 가질 때만 상수 — **센서값에 0은 평균 왜곡 위험**

CODE

```
# 숫자 결측이 "참여 안 함"을 의미하면 0
df["행사참여"] = df["행사참여"].fillna(0)

# 글자 결측은 "미상"."확인필요" 같은 표시값
df["설비명"] = df["설비명"].fillna("미상")

# 주의: 센서값에 0을 채우면 평균이 왜곡됨
```

상수 대체가 어울리는 경우 — 결측이 "없음"."0"."미상" 같은 특정 의미를 가질 때
온도가 비었다고 0을 채우면 "온도 0도"라는 틀린 정보가 되어 **평균이 엉뚱하게 낮아짐**
측정값은 0 같은 상수보다 평균이나 중앙값으로 채우는 게 일반적

평균으로 채우기

그 컬럼의 평소 대표값으로 채우는 측정값 결측의 표준 — **이상치에 약함**

01 · 사용법

멀쩡한 값들의 평균으로 채움

```
m = df["센서09"].mean()  
df["센서09"] = \  
df["센서09"].fillna(m)
```

판다스는 결측을 알아서 빼고 평균 계산

02 · 약점

극단값에 휘둘리는 한계

**유난히 크거나 작은
이상치에 휘둘림**

왜곡된 평균으로 빈 칸을 채움

평균 계산 전 위장 결측을 NaN으로 먼저 변환 필수

중앙값으로 채우기

크기순 한가운데 값 — 이상치가 있어도 흔들리지 않는 안전한 대체

01 · 정의

크기순 줄 세웠을 때 한가운데

1, 2, 3, 4, 100

평균 22 · **중앙값 3**

데이터의 진짜 중심에 훨씬 가까움

02 · SECOM 센서09

평균과 중앙값의 큰 차이

평균 116

중앙값 95

유난히 큰 값들이 평균을 끌어올림

03 · CODE

mean을 median으로 교체만

```
m = df["센서09"].median()
```

```
df["센서09"] = \  
df["센서09"].fillna(m)
```

치우치거나 이상치 의심 시 안전

평균 116으로 채우면 중심보다 높게, 중앙값 95로 채우면 진짜 중심에 가깝게 채워짐

평균 vs 중앙값, 무엇을 쓸까

내 데이터 모양에 맞는 걸 고르는 안목 — 둘 다 중심값이지만 성격이 다름

구분	평균 대체	중앙값 대체
이상치 영향	✖ 크게 휘돌림	✔ 거의 영향 없음
적합 분포	고른 분포	치우친 분포
예시	안정적 온도·압력	SECOM 센서09 (평균 116·중앙값 95)
생활 예	사장 포함 평균 천만 원	실제 형편 300만 원

평균이 나쁜 게 아님 — 이상치가 없고 고르게 퍼졌으면 평균이 가장 자연스러운 대표값

분포를 보고 고르는 법

대체 전 평균과 중앙값을 둘 다 구해 비교 — 한 번의 비교가 대체 품질을 좌우

STEP 1

둘 다 구하기

대체하기 전 평균과 중앙값을 모두 계산해 나란히 비교.

mean, median



STEP 2

비슷하면 평균

두 값이 비슷하면 분포가 고르다는 뜻 — 평균을 자연스러운 대표값으로 선택.

안정적 온도·압력



STEP 3

많이 다르면 중앙값

눈에 띄게 다르면 이상치 신호 — 안전한 중앙값을 보험처럼 선택.

센서09 (116 vs 95)

헛갈릴 때는 중앙값 — 미처 못 걸러낸 이상치가 남아 있어도 영향을 거의 안 받는 보험

최빈값으로 채우기 — 범주형

글자·분류 데이터는 평균이 없어 **가장 자주 나타나는 값**으로 채움

01 · 범주형 정의

평균을 낼 수 없는 데이터

설비명·등급·작업조
글자·분류 데이터

1, 2, 3 같은 등급 숫자도 사실은 범주

02 · AI4I 제품유형

분포로 본 최빈값

L 119 · M 55 · H 26
최빈값은 L

제품유형 결측은 L로 채우면 합리적

03 · CODE

숫자는 평균·중앙값, 범주는 최빈값

```
top = df["제품유형"] \
      .mode()[0]
```

```
df["제품유형"] = \
df["제품유형"] \
      .fillna(top)
```

너무 많이 채우면 다양성 손실 주의

앞 값으로 채우기 — ffill

설비 상태는 짧은 시간에 급변하지 않음 — 바로 앞 값으로 잇는 시계열의 자연스러운 대체

CODE

```
# 시간 순서대로 먼저 정렬
df = df.sort_values("측정시각")

# 결측을 바로 앞의 멀쩡한 값으로 메움
df["온도"] = df["온도"].ffill()

# 73, NaN, NaN, 75 → 73, 73, 73, 75
```

10시 73도 · 11시 NaN → 11시를 73도로 채움 — 직전 멀쩡한 값을 끌어와 메움
최신 판다스에서는 ffill을 직접 호출 — 코드도 간결하고 의도도 분명
결측이 길게 이어지면 같은 값이 여러 칸 반복 — 잠깐 빠진 한두 칸에 적합

뒤 값으로 채우기 — bfill

맨 앞이 비어 ffill로 못 채울 때의 보완 — ffill 먼저, 남은 앞부분을 bfill로 마무리

01 · 작동

뒤쪽 값을 당겨 옴

10시 NaN · 11시 75도
10시를 75도로 채움

ffill과 방향만 다르고 원리는 동일

02 · 기본은 ffill

우선순위 판단

과거 값으로 현재 추측이
미래 값보다 자연스러움

ffill을 기본으로 쓰는 이유

03 · 조합 사용

ffill 후 남은 맨 앞을 bfill

```
df["온도"] = (  
    df["온도"]  
    .ffill()  
    .bfill()  
)
```

두 방향에서 채워 어느 위치도 놓치지 않음

ffill·bfill 사용 시 주의점

인접 값이 비슷하다는 가정 — **정렬·그룹·결측 길이** 세 가지 점검 필수

01 · 정렬 필수

순서가 의미 있어야 함

**쓰기 전 반드시
시간 컬럼으로 정렬**

뒤죽박죽 상태에선 아무 상관없는 값을 끌어옴

02 · 긴 결측 주의

같은 값 반복의 위험

**결측이 연달아 있으면
같은 값이 반복**

잠깐 빠진 한두 칸에만 적합, 오래 끊긴 구간은
흐름 왜곡

03 · 그룹 분리

설비 간 값 섞임 방지

**설비별로 묶은 뒤
그룹 안에서만 ffill**

A 설비 빈칸을 B 설비 값으로 채우는 사고 방지

그룹별로 채우기 — groupby

같은 종류끼리 값이 비슷할 때 **한 덩어리 평균보다 그룹별 평균이 더 정확**

CODE

```
# 제품유형(L/M/H)으로 묶어 그 그룹의 평균으로 빈 칸 채움
df["토크"] = df.groupby("제품유형")["토크"].transform(
    lambda x: x.fillna(x.mean())
)

# L그룹 빈칸은 L그룹 평균, M그룹 빈칸은 M그룹 평균으로
```

프레스의 빈 온도를 절삭기까지 섞인 전체 평균으로 채우는 건 부정확

설비명으로 묶으면 프레스끼리·절삭기끼리 모이고, 프레스 빈 온도는 프레스 평균으로 채움
그룹이 너무 잘게 나뉘어 값이 거의 없으면 그 그룹 평균이 불안정 — 주의

대체가 분포에 미치는 영향

채운다는 건 없던 값을 만들어 넣는 일 — **분포가 인위적으로 뽀족해짐**

01 · 데이터 쓸림

한 지점에 뭉치는 효과

**결측 100개를 평균으로
채우면 그 값에 100개 몰림**

원래 다양하게 퍼져 있어야 할 값들이 한곳에
뭉침

02 · 분포 변형

뽀족해지고 다양성 손실

**평균 근처가
비정상적으로 뽀족**

흩어진 정도(다양성)가 실제보다 줄어듦

03 · 안전 기준

채울수록 왜곡 심해짐

**결측 절반 넘는 컬럼은
차라리 제거가 나음**

대체 = 완벽한 복원이 아닌 합리적 추측

inplace 파라미터 주의

"처리했는데 왜 안 바뀌지"의 십중팔구 — **결과를 변수에 안 담아서** 생기는 문제

방식	❌ 동작	✅ 권장 방식
결과 미저장	화면엔 처리된 듯 보이지만 원본은 그대로	반드시 변수에 담기
inplace=True 사용	결과를 또 변수에 담으면 텅 빈 값	아예 사용하지 않기
최신 판다스 권고	inplace 권장 안 함 (Deprecated 흐름)	한 가지 원칙으로 통일

우리 원칙은 **하나** — **처리 결과는 항상 변수에 담는다**
`df = df.fillna(0)` 또는 `clean = df.fillna(0)`

처리 후 검증과 저장

처리는 확인으로 시작해 확인으로 끝남 — **isna 검증 후 새 이름으로 저장**

CODE

```
# 1) 결측치 정말 사라졌는지 검증
print(df.isna().sum().sum()) # 0 이면 처리 완료

# 2) 새 이름으로 저장 - 원본은 보존
df.to_csv("secom_처리완료.csv",
         index=False,
         encoding="utf-8-sig")
```

index=False — 행 번호가 안 들어가게 / **encoding="utf-8-sig"** — 한글 안 깨지게
처리가 잘못됐을 때 돌아갈 수 있도록 **원본에 덮어쓰지 말고 새 이름으로 저장**
글자 컬럼의 공백처럼 못 잡은 결측치 남기도 함 — 검증에서 0이 아니면 다시 살피기

결측치 처리 흔한 실수

공통점은 모두 **확인하지 않고 서두르는 데서 옴**

#	실수	증상	예방
1	결과 미반영	처리했는데 결측이 그대로	결과를 반드시 변수에 담기
2	위장 결측 방치	0~999 두고 평균 → -130 망가짐	먼저 NaN으로 변환
3	과한 대체	절반 넘는 결측을 다 채움	차라리 컬럼 제거
4	정답 함부로 채움	불량여부 같은 정답을 추측으로 채움	subset으로 빈 행만 제거
5	검증 건너뛰기	처리 후 isna로 확인 안 함	isna().sum().sum() == 0 확인

처리 전 확인 → 신중히 선택 → 처리 후 검증 — 이 순서만 지키면 대부분 피할 수 있는 함정

대체 방법 선택 기준

"이 빈 칸에 가장 그럴듯한 값은 무엇일까" — 정답 외우기보다 질문하는 습관

데이터 종류·상황	선택	이유
글자·분류 (범주형)	최빈값	평균이 없어 가장 자주 나타나는 값으로
숫자 측정값, 고른 분포	평균	이상치 없고 고르면 가장 자연스러운 대표값
숫자 측정값, 치우친 분포	중앙값	이상치에 흔들리지 않는 안전한 보험
시간·순서 있는 데이터	ffill + bfill	직전 값으로 잇고 맨 앞은 뒤 값으로 보완
의미 있는 결측 (없음·0)	상수	참여안함=0, 미상="미상" 등 명시적 표시
그룹이 뚜렷한 데이터	그룹별 평균·중앙값	같은 종류끼리 묶어 채우면 더 정확

컬럼마다 다른 방법, 채우고 확인

데이터 전체에 한 가지 대체를 일괄 적용하지 않음 — 컬럼별 정석

STEP 1

종류·분포 확인

컬럼별로 종류와 분포를 하나씩 확인. 데이터를 깊이 이해하는 시간.

dtype · mean · median



STEP 2

알맞은 방법으로 채움

컬럼마다 가장 알맞은 방법으로 채움. 한 가지를 일괄 적용하지 않음.

fillna · ffill · groupby



STEP 3

범위·분포 재확인

채운 값이 정상 범위인지, 분포가 일그러지지 않았는지 확인.

describe · hist

대체는 한 번에 끝내는 게 아니라 채워 보고 확인하고 다듬는 과정

결측치 처리 전체 워크플로우

제거로 덜고 대체로 채움 — 확인 → 판단 → 처리 → 검증·저장

STEP 1

확인

위장 결측을 NaN으로 변환. 비율·순위·분포 파악.

isna · mean · sort_values



STEP 2

판단

컬럼·행별로 어떻게 처리할지 방향을 결정.

제거 vs 대체



STEP 3

처리

제거로 덜고 대체로 채움. 컬럼별 알맞은 방법 적용.

drop · fillna · ffill



STEP 4

검증·저장

결측 0 확인. 새 파일로 저장하며 처리 내역 기록.

to_csv

앞 시간의 확인부터 이번 모듈의 제거·대체까지, 하나의 실전 흐름으로 짚어 보기

이 흐름은 모든 전처리의 뼈대

이상치 처리에서도, 그 뒤 모든 전처리에서도 똑같이 반복되는 **데이터 작업의 기본 리듬**

01 · 반복 과정

한 번에 완벽하지 않음

**점점 다듬어 가는
반복 과정**

이상하면 다시 돌아가 방법을 바꿈

02 · 기록 보존

되돌아오기·설명하기 위해

**무엇을 어떻게 했는지
기록으로 남기기**

수정·인계 모두 쉬워짐

03 · 확장성

이 다섯 단계는 결측 전용 아님

**이상치·정규화
모든 전처리에 동일 적용**

지금 익히면 어떤 전처리든 뼈대를 갖춘 셈

실습 5. fillna 평균·중앙값 대체

결측을 평균과 중앙값으로 채우고 차이 이해

목표

결측을 평균과 중앙값으로 채우고 차이 이해

단계

- 대상 컬럼의 평균과 중앙값을 각각 구해 비교
- fillna로 평균을 채운 결과 만들기
- fillna로 중앙값을 채운 결과 만들기(이상치에 강함)

예상 결과

센서17 평균 466.26·중앙값 465.9로 대체, 남은 결측 0

실습 6. 최빈값·앞뒤 값 대체

범주형은 최빈값, 시계열은 앞뒤 값으로 채우기

목표

범주형은 최빈값, 시계열은 앞뒤 값으로 채우기

단계

- 범주형 열의 최빈값을 구해 채우기
- 측정시각 순으로 정렬해 시계열 순서 만들기
- ffill로 앞 값, bfill로 남은 앞쪽 결측까지 채우기

예상 결과

설비명은 최빈값(절삭기A), 온도는 앞뒤 값으로 대체

실습 7. 그룹별 대체

그룹별 평균으로 채워 집단 특성 반영

목표

그룹별 평균으로 채워 집단 특성 반영

단계

- 제품유형으로 그룹을 나누기
- 각 그룹의 평균으로 그 그룹의 결측을 채우기
- 남은 수치 결측은 전체 중앙값으로 마무리하고 검증

예상 결과

토크를 유형별 평균으로 대체, 남은 결측 0

실습 8. 제거 vs 대체 비교

같은 데이터에 제거와 대체를 적용해 결과 비교

목표

같은 데이터에 제거와 대체를 적용해 결과 비교

단계

- 결측 심한 컬럼을 먼저 뺀 기준 데이터 만들기
- 기준 데이터에서 결측 행을 삭제한 제거 버전 만들기
- 기준 데이터의 결측을 중앙값으로 채운 대체 버전 만들기

예상 결과

제거 버전 110행, 대체 버전 250행(모두 유지)

실습 9. SECOM·AI4I 종합 처리

제거와 대체를 조합해 전체 결측을 처리하고 저장

목표

제거와 대체를 조합해 전체 결측을 처리하고 저장

단계

- 결측 비율 높은 컬럼을 제거하고 나머지는 중앙값으로 채우기
- 처리 후 남은 결측과 크기를 확인하고 파일로 저장
- 같은 절차를 AI4I 데이터에도 반복해 결측 0 확인

예상 결과

SECOM 결측 0·저장, AI4I 결측 0